



Università degli Studi di Salerno

Dipartimento di Ingegneria Elettrica e Matematica Applicata

Corso di Laurea in Ingegneria Informatica Insegnamento di Intelligenza Artificiale:
Metodi e Applicazioni — A.A. 2025/2026

Progettazione di un Agente di Guida Autonoma su simulatore TORCS

Implementazione e valutazione tramite Behavioral Cloning con algoritmo K-Nearest Neighbors

Studente: Mario Cucciniello (Matr. 0612709473)

Professori:

- Prof. Vento Mario
- Prof.ssa Saggese Alessia

Indice

1. Introduzione

2. Raccolta e Preparazione dei Dati

- 2.1 Setup e Processo di Raccolta (UDP Telemetria)
- 2.2 Selezione dei Giri e Bilanciamento delle Traiettorie
- 2.3 Analisi Descrittiva del Dataset

3. Addestramento del Modello KNN

- 3.1 Preprocessing e Standardizzazione delle Feature
- 3.2 Architettura del Modello KNN (KNeighborsRegressor)
- 3.3 Valutazione Prestazionale sul Test Set (Metriche R2, MAE, RMSE)

4. Guida Autonoma in Tempo Reale

- 4.1 Ciclo di Controllo UDP e Costruzione dello Stato
- 4.2 Ottimizzazioni Fisiche e Controlli Attivi
 - 4.2.1 Moltiplicatori di Controllo (Steer & Brake Boost)
 - 4.2.2 ABS & Traction Control (TCS) Rule-Based
 - 4.2.3 Cambio Marcia Automatico con Blocco in Curva
 - 4.2.4 Frenata Preventiva per l'Ingresso Chicane

5. Risultati e Conclusioni

1. Introduzione

Il presente progetto descrive la progettazione e l'implementazione di un agente di guida autonoma in grado di condurre un veicolo da corsa all'interno del simulatore open-source **TORCS (The Open Racing Car Simulator)**. La metodologia adottata appartiene alla famiglia dell'**Imitation Learning** e, in particolare, consiste nella tecnica del **Behavioral Cloning (BC)**.

Nel Behavioral Cloning, l'agente apprende la funzione di controllo (policy) comportandosi in modo puramente reattivo (mapping diretto da stato sensoriale ad azioni di controllo) imitando le decisioni precedentemente registrate da un pilota umano. A differenza del *Reinforcement Learning*, che richiede un addestramento online per tentativi ed errori estremamente lungo e computazionalmente oneroso (spesso distruttivo per il motore fisico nelle fasi iniziali), il Behavioral Cloning offre il vantaggio di una convergenza offline immediata e garantisce che l'agente sposi fin da subito le traiettorie ideali (racing line) impostate dall'esperto.

La pipeline di sviluppo si articola in tre fasi fondamentali:

1. **Raccolta dati:** Registrazione della telemetria sensoriale e dei comandi di guida (sterzo, acceleratore, freno) durante sessioni di guida manuale su tracciato.
2. **Addestramento:** Pre-elaborazione, bilanciamento del dataset e fitting di un regressore spaziale **K-Nearest Neighbors (KNN)**.
3. **Guida autonoma:** Controllo a ciclo chiuso tramite pacchetti UDP scambiati a 50Hz, integrando moduli di post-processing fisici (ABS, TCS, staccata preventiva della chicane) per superare i limiti strutturali di smoothing del modello matematico.

2. Raccolta e Preparazione dei Dati

2.1 Setup e Processo di Raccolta

La telemetria di guida è stata raccolta pilotando manualmente la vettura sul circuito tramite periferica di controllo analogica. Durante il moto, il client scambia messaggi con il server TORCS ad una frequenza di 50Hz (ogni 20 ms). Ad ogni tick della simulazione, lo stato fisico della vettura viene campionato in un vettore di **24 feature**:

- `angle`: Angolo di orientamento della vettura rispetto all'asse della pista (in radianti).
- `trackPos`: Scostamento laterale dal centro della pista (compreso tra -1 e $+1$, dove 0 indica la mezzzeria e ± 1 i cordoli esterni).
- `speedX`: Velocità longitudinale della vettura (in km/h).
- `speedY`: Velocità trasversale della vettura (in km/h).
- `wsv_avg`: Velocità angolare media di rotazione delle ruote (derivata da `wheelSpinVel`).
- `track_0` $\dots$ `track_18`: 19 sensori laser distanziometrici che misurano la distanza dal limite della pista in un raggio visivo di 200 metri, spazati angularmente di 5° l'uno dall'altro nell'arco frontale $[-90^\circ, +90^\circ]$.

I target associati che l'agente deve apprendere sono le tre azioni fondamentali prodotte dal pilota:

1. `steer`: Comando di sterzo (compreso nel range $[-1.0, 1.0]$, dove valori negativi indicano svolta a destra).
2. `accel`: Pressione dell'acceleratore (range $[0.0, 1.0]$).
3. `brake`: Pressione del freno (range $[0.0, 1.0]$).

I dati di ogni giro sono stati inizialmente salvati in formato CSV all'interno della cartella `dataset_laps/` per essere successivamente letti ed elaborati in blocco.

2.2 Selezione dei Giri e Bilanciamento delle Traiettorie

Per garantire che il KNN generalizzasse in modo robusto anche di fronte ad errori di traiettoria, il dataset include sia giri condotti lungo la traiettoria ideale (racing line ottimale a velocità elevata), sia giri "conservativi" o di "recovery", in cui sono state registrate le manovre necessarie per rientrare in pista a seguito di sbandate o collisioni.

Uno dei problemi principali del Behavioral Cloning applicato alla guida è lo **sbilanciamento intrinseco delle classi**. In un circuito tipico, i tratti in rettilineo in cui l'acceleratore è premuto al massimo (`accel = 1.0`, `brake = 0.0`, `steer = 0.0`) rappresentano oltre il 70-80% del tempo totale di guida. Se addestrato su dati grezzi, il KNN tenderebbe a predire accelerazione massima e zero freni anche all'approssimarsi delle curve, tirando dritto.

Per risolvere questo problema, lo script `preparazione_dataset.py` implementa una logica di **bilanciamento e sottocampionamento selettivo**:

- I record che presentano accelerazione costante in rettilineo e assenza di frenata vengono decimati sistematicamente.
- Vengono mantenuti intatti tutti i transitori di frenata e di inserimento in curva, rinviorendo artificialmente la percentuale di campioni in cui `brake > 0` e `abs(steer) > 0.05`.
- Questo bilanciamento garantisce che i "vicini spaziali" selezionati dal KNN all'ingresso

di una curva riflettano le decisioni di decelerazione e non le fasi di percorrenza veloce sul dritto.

2.3 Analisi Descrittiva del Dataset

Il dataset finale consolidato è composto da circa **55.000 frame** di telemetria puliti. La distribuzione percentuale dei dati e i parametri medi dello sterzo evidenziano che il circuito presenta zone con dinamiche fortemente contrastanti:

Zona del Circuito	Incidenza % Frame	Velocità Media (km/h)	Steer Medio	Steer Dev. Std.
Partenza / Allineamento	12.5%	120	+ 0.002	0.045
Rettilinei Veloci	42.1%	195	-0.005	0.021
Curve Veloci (Raggio Ampio)	18.3%	145	+ 0.085	0.120
Tornanti Stretti	15.2%	85	+ 0.220	0.280
Chicane in Discesa (Corkscrew)	11.9%	72	-0.180	0.315

L'elevata deviazione standard dello sterzo nelle vicinanze della chicane riflette le repentine correzioni di traiettoria necessarie per impostare i cambi di direzione destra-sinistra.

3. Addestramento del Modello KNN

3.1 Preprocessing e Standardizzazione delle Feature

Le feature estratte presentano scale fisiche profondamente disomogenee: le velocità variano tra \$0\$ e \$220\$ km/h, le distanze dei sensori laser spaziano da \$0\$ a \$200\$ metri, mentre l'angolo e la posizione laterale assumono valori decimali molto piccoli. Poiché l'algoritmo KNN calcola la distanza geometrica tra i punti, feature con scale numeriche più ampie dominerebbero il calcolo della distanza a scapito di quelle più piccole ma fisicamente critiche (come angle).

Per ovviare a ciò, viene applicata una pipeline di standardizzazione tramite `StandardScaler`, che trasforma ciascuna feature x sottraendo la media μ e dividendo per la deviazione standard σ :
$$z = \frac{x - \mu}{\sigma}$$

In questo modo, ogni feature assume media nulla ($\mu = 0$) e varianza unitaria ($\sigma^2 = 1$), garantendo un peso identico nel calcolo della vicinanza.

3.2 Architettura del Modello KNN

Il modello di machine learning utilizzato è un **K-Neighbors Regressor** (`KNeighborsRegressor` di `scikit-learn`) configurato con i seguenti iperparametri:

- `n_neighbors = 3`: Il valore di $K=3$ è stato scelto sperimentalmente per bilanciare la capacità di generalizzazione e la reattività locale del modello. Valori di K troppo alti causerebbero un eccessivo smoothing (tagliando le curve), mentre $K=1$ renderebbe la guida instabile ed estremamente sensibile al rumore nei dati.
- `weights = 'distance'`: I comandi predetti sono calcolati come media ponderata delle azioni dei 3 vicini, dando un peso maggiore ai campioni che presentano una distanza minore rispetto allo stato corrente.
- `algorithm = 'ball_tree'`: Struttura dati ad albero multidimensionale che raggruppa i dati di training in ipersfere nidificate. Questa indicizzazione riduce la complessità di ricerca a runtime da lineare $O(N)$ a logaritmica $O(\log N)$, permettendo al client di completare l'inferenza in meno di 0.5 ms ed evitando timeout di connessione UDP.
- `metric = 'euclidean'`: La distanza d tra lo stato di query $\mathbf{u}$ e un punto del dataset $\mathbf{v}$ è calcolata tramite metrica euclidea su $D=24$ dimensioni:
$$d(\mathbf{u}, \mathbf{v}) = \sqrt{\sum_{i=1}^D (u_i - v_i)^2}$$

3.3 Valutazione Prestazionale sul Test Set

Il dataset bilanciato è stato suddiviso in un **80% per il training set** e un **20% per il test set**. Le prestazioni del modello sulle tre variabili di controllo sono descritte nella tabella seguente tramite il coefficiente di determinazione (R^2), l'errore assoluto medio (MAE) e la radice dell'errore quadratico medio (RMSE):

- **R2 Score**: Coefficiente di Determinazione
- **MAE**: Errore Assoluto Medio
- **RMSE**: Radice dell'Errore Quadratico Medio

Target di Controllo	R2 Score	MAE	RMSE
Sterzo (<code>target_steer</code>)	0.863	0.049	0.130

Acceleratore (target_accel)	0.837	0.076	0.176
Frenata (target_brake)	0.829	0.020	0.098

I risultati evidenziano un'ottima accuratezza, in particolare sulla frenata e sullo sterzo. L'errore leggermente superiore sull'acceleratore è causato dalla rapidità transitoria con cui il pilota rilascia o preme il pedale in uscita di curva.

4. Guida Autonoma in Tempo Reale

4.1 Ciclo di Controllo UDP e Costruzione dello Stato

A runtime, lo script `guida_autonoma_knn.py` si connette a TORCS tramite socket UDP sulla porta 3001. Ad ogni iterazione del loop:

1. Riceve la stringa XML contenente i sensori di bordo.
2. Costruisce il vettore di stato ordinato di 24 elementi. In caso di letture laser mancanti, viene assegnato il valore di default di \$200.0\$ metri (assenza di ostacoli).
3. Applica la normalizzazione tramite lo scaler caricato.
4. Esegue l'**inferenza KNN** per predire i valori grezzi di sterzo, acceleratore e freno.
5. Sottopone le azioni predette ad una serie di controlli fisici e logiche di post-processing prima di trasmettere il pacchetto a TORCS.

4.2 Ottimizzazioni Fisiche e Controlli Attivi

A causa dell'effetto di smoothing spaziale intrinseco dell'algoritmo KNN (che calcola una media locale tra vicini), le azioni di controllo predette tendono ad essere più blande rispetto a quelle reali (es. una frenata decisa a \$1.0\$ viene mitigata a \$0.4\$, e uno sterzo a \$0.8\$ scende a \$0.5\$). Per compensare questo fenomeno e garantire una guida sicura e veloce, sono state introdotte le seguenti logiche fisiche nel loop di controllo.

4.2.1 Moltiplicatori di Controllo (Steer & Brake Boost)

I comandi grezzi predetti dal modello KNN vengono amplificati tramite opportuni moltiplicatori calibrati su pista e successivamente limitati entro i range di sicurezza tramite saturazione (`clip`):

$$\begin{aligned} \text{steer}_{\text{applicato}} &= \text{clip}(\text{steer}_{\text{KNN}} \times 1.2, -1.0, 1.0) \\ \text{brake}_{\text{applicato}} &= \text{clip}(\text{brake}_{\text{KNN}} \times 2.5, 0.0, 1.0) \end{aligned}$$

Il moltiplicatore dello sterzo a **1.2** elimina il sottosterzo nelle curve a raggio stretto, mentre il moltiplicatore del freno a **2.5** assicura che l'auto decelererà con la massima forza nelle staccate violente.

4.2.2 ABS & Traction Control (TCS) Rule-Based

I sistemi di controllo attivo sfruttano la velocità angolare delle singole ruote (ω_i), letta dal sensore `wheelSpinVel`) e la velocità longitudinale del veicolo per prevenire perdite di aderenza:

- **Traction Control (TCS):** Se le ruote motrici posteriori slittano significativamente rispetto alle anteriori trascinate, la coppia motrice viene dimezzata per recuperare aderenza:
$$\text{Se } (\omega_{\text{ruote_post}} - \omega_{\text{ruote_ant}}) > 15 \implies \text{accel} = \text{accel} \times 0.5$$
- **Anti-lock Braking System (ABS):** Se il freno è attivo ad una velocità superiore a 15 km/h e la rotazione media delle ruote anteriori scende sotto la soglia di bloccaggio (5 rad/s), la forza frenante viene ridotta al 10% per consentire al pilota di sterzare:
$$\text{Se } (\text{brake} > 0.1) \wedge (\text{speedX} > 15) \wedge (\omega_{\text{ruote_ant_media}} < 5) \implies \text{brake} = \text{brake} \times$$

4.2.3 Cambio Marcia Automatico con Blocco in Curva

Il cambio marcia è gestito da una logica sequenziale basata su soglie di velocità predefinite per ciascuna delle 6 marce. Tuttavia, per evitare cambi marcia o scalate destabilizzanti durante le forti accelerazioni laterali (che causerebbero sbandate), la marcia viene bloccata all'interno delle curve strette: $\text{Se } |\text{steer}| > 0.4 \implies \text{mantiene la marcia corrente}$

4.2.4 Frenata Preventiva per l'Ingresso Chicane

La chicane del Corkscrew rappresenta il punto più critico del tracciato. A causa della pendenza e dell'angolo cieco, le predizioni grezze del KNN non avviavano la frenata con sufficiente anticipo, portando l'auto ad andare dritta contro i muretti esterni.

È stata implementata una regola di **frenata preventiva mirata**, attiva solo in rettilineo ad alta velocità all'approssimarsi della chicane (valutando il sensore laser anteriore `track_9`): $\text{Se } (\text{speedX} > 70) \wedge (|\text{steer}| < 0.05) \wedge (\text{track}_9 < 20.0) \implies \begin{cases} \text{brake} = \max(\text{brake}, 0.7) \\ \text{accel} = 0.0 \end{cases}$

Non appena il veicolo inizia la svolta impostando la traiettoria ($|\text{steer}| \geq 0.05$), la regola si disattiva istantaneamente, lasciando che l'auto scorra fluida e veloce all'interno della chicane senza inutili rallentamenti in percorrenza.

5. Risultati e Conclusioni

L'applicazione della pipeline di Behavioral Cloning basata su K-Nearest Neighbors ha prodotto risultati eccellenti. L'agente intelligente ha dimostrato di aver appreso in modo robusto le traiettorie del circuito Corkscrew, completando sessioni di gara continue senza collisioni o uscite di pista.

Riscontro Cronometrico

Grazie alla sinergia tra la precisione spaziale del KNN e l'implementazione dei controlli dinamici attivi (ABS, TCS, moltiplicatori e staccata preventiva della chicane), l'auto ha registrato un **tempo record sul giro singolo di 01 : 22 . 56**. Questo risultato rappresenta un miglioramento netto rispetto ai modelli classici sprovvisti di aiuti fisici (che faticano a scendere sotto il minuto e 27 secondi a causa del sottosterzo indotto dallo smoothing).

In conclusione, il progetto dimostra come il Behavioral Cloning, sebbene limitato strutturalmente da fenomeni di covariate shift e attenuazione delle risposte dinamiche, possa essere reso estremamente efficiente e competitivo affiancandolo a logiche fisiche di post-processing. L'utilizzo dell'algoritmo KNN con indicizzazione Ball Tree si è rivelato vincente per la stabilità in tempo reale, offrendo una valida e immediata alternativa ai lunghi cicli di addestramento dell'Apprendimento per Rinforzo.